01

MGU BCA - 3RD SEM

Database Management Systems

Module 1 - Introduction to Databases

By Aben Sojan

Exam - 6 Nov 2026

Complete question bank: every important, repeated and sure-mark question for this module, each with a full-mark model answer. Nothing important left out.

EXAM PATTERN (from real MGU question papers)

MGU BCA Honours DBMS (MG3CCRBCA201): 70 marks, 2 hours. Part A: 5 x 2 marks, short questions from all modules. Part B: answer any 5 of 6-mark questions. Part C: answer any 2 of 15-mark essays. Old CBCS papers (2019-2021): 80 marks, 3 hours, any-10 x 2 + any-6 x 5 + any-2 x 15. Part C hotspots: COMPANY ER diagram, Normal Forms essay, anomalies.

TOPIC 1

Data, Database and DBMS

1

Data means raw facts and figures, like names, marks or prices, with no meaning by themselves.

A database is an organised collection of related data stored together, like all student records of a college.

DBMS (Database Management System) is software that stores, manages and retrieves data from a database.

Examples of DBMS: MySQL, Oracle, PostgreSQL and MongoDB.

QUESTION BANK

1 Define data, database and DBMS.

IMPORTANT

Data means raw, unprocessed facts and figures that have no meaning by themselves, like a name, a mark or a price.

For example, the values 'Aben', 85 and 'Kottayam' are data, but they tell us nothing until they are organised.

A database is an organised collection of related data stored together so that it can be used easily.

For example, all the student records of a college, with names, roll numbers and marks, form a student database.

A DBMS (Database Management System) is software that creates, stores, updates and retrieves data from a database.

It acts as an interface between the users and the database, so users do not need to know how data is stored on disk.

A DBMS also manages security, backup and sharing of data among many users at the same time.

Examples of DBMS software are MySQL, Oracle, PostgreSQL and MongoDB.

2 **What is a DBMS? Give two examples.**

IMPORTANT

A Database Management System (DBMS) is a software system that enables users to create, store, retrieve, update and manage data in databases in an organised and efficient way.

It works as a bridge between the users and the physical database, so users can ask for data in a simple language like SQL without knowing how it is stored.

A DBMS allows many users to access the same data at the same time without conflicts, using locking and concurrency control.

It provides security through user accounts, passwords and access permissions, so only authorised users can see or change sensitive data.

It keeps data consistent and correct using constraints like primary key and foreign key, which block invalid entries.

It includes backup and recovery tools, so data can be restored if the system crashes or a disk fails.

Two popular examples of DBMS are MySQL, which is widely used in web applications, and Oracle, which is used in large enterprise systems.

Other well-known examples include PostgreSQL and MongoDB.

TOPIC 2

Database Applications

2

Banking: customer accounts, transactions and ATM records.

Education: student admissions, marks and attendance.

Railways and airlines: ticket booking and seat availability.

E-commerce: products, orders and customers, like Flipkart or Amazon.

Hospitals: patient records, doctors and appointments.

QUESTION BANK

3 List any four applications of databases with one line each.

IMPORTANT

Banking: a DBMS stores customer account details and records every deposit, withdrawal and ATM transaction safely.

Education: colleges use a DBMS to manage student admissions, marks, attendance and fees in one place.

Railway reservation: a DBMS books tickets and tracks seat availability live for thousands of passengers at once.

E-commerce: websites like Flipkart and Amazon use a DBMS to manage products, orders, payments and customer details.

Hospitals: a DBMS stores patient records, doctor details and appointment schedules so that any doctor can see a patient's history quickly.

Libraries: a DBMS tracks books, members and issue and return dates, making searching and lending simple.

These applications are possible because a DBMS handles large amounts of shared data quickly and reliably.

4 Why do banks and railways prefer DBMS over plain files?

IMPORTANT

Banks and railways prefer a DBMS over plain files because thousands of users need to access and update the same data at the same time safely.

A DBMS supports concurrent access with locking, so two clerks booking tickets or updating accounts never overwrite each other's work.

Every transaction is recorded completely through the ACID properties, so there are no half-written updates like money debited but not credited.

Data stays consistent everywhere: once a ticket is booked, it instantly becomes unavailable to every other user.

A DBMS enforces rules and access control, so only authorised staff can see or change sensitive records like account balances.

Regular backups and recovery tools protect years of records from being lost in a crash.

Plain files cannot offer any of this, so they become unmanageable and unsafe at the scale of banks and railways.

TOPIC 3

3

Advantages of DBMS

Reduces data redundancy: the same data is not stored in many separate files.

Data consistency: one update reflects everywhere, so data never mismatches.

Data sharing: many users can use the same data at the same time.

Data security: passwords and access control protect sensitive data.

Data integrity: rules keep data correct, like marks always between 0 and 100.

Backup and recovery: data can be restored after a crash.

QUESTION BANK

5 **Explain the advantages of DBMS over the traditional file system. (Oct 2019, Dec 2021)**

REPEATED

The traditional file system stores data in separate files for each application, while a DBMS stores it once in a central shared database, which gives many advantages.

Less redundancy: the same data is not repeated in many files, so storage is not wasted and there is only one copy to maintain.

Data consistency: because there is one shared copy, an update made once is reflected everywhere, so copies never mismatch.

Data sharing: many users and programs can use the same data at the same time through controlled concurrent access.

Data security: user accounts, passwords and access permissions protect sensitive data from unauthorised users.

Data integrity: rules like primary key, foreign key and CHECK constraints keep data correct, for example marks always stay between 0 and 100.

Backup and recovery: the DBMS can take automatic backups and restore data after crashes, hardware failures or power cuts.

Easy querying: languages like SQL let users get answers quickly without writing complex file-handling programs.

6 What is data redundancy? How does DBMS reduce it?

IMPORTANT

Data redundancy means the same data is stored again and again in different files, for example a student's address repeated in the admission file, the library file and the hostel file.

It wastes storage space and causes inconsistency, because if one copy is updated and another is not, the copies disagree with each other.

A DBMS reduces redundancy by storing data once in a central database that every application and user shares.

Instead of repeating the address in every department's file, it is stored once in a Student table and all departments read it from there.

When the address changes, it is updated in one place and the change is instantly visible to everyone.

This is why redundancy in a DBMS is controlled and minimal, unlike in the traditional file system where duplication grows out of control.

TOPIC 4

Disadvantages of DBMS

4

High cost: DBMS software, hardware and training are expensive.

Needs trained staff like a DBA to manage it.

Large and complex: needs more memory and processing power.

Single point of failure: if the DBMS goes down, all applications stop.

QUESTION BANK

7 What are the disadvantages of DBMS?

IMPORTANT

High cost: DBMS software licences, powerful hardware and training for staff make it expensive to set up and run.

Large and complex: a DBMS needs much more memory, disk space and processing power than simple files, and it is harder to manage.

Needs trained staff: a skilled DBA (Database Administrator) is required to install, secure, tune and maintain the system.

Single point of failure: if the central DBMS goes down, every application that depends on it stops working at the same time.

Performance overhead: features like locking, logging and security checks add extra work, which can slow down very simple tasks.

Conversion effort: moving old file-based data into a DBMS takes time, planning and careful data cleaning.

8 DBMS has disadvantages, yet organisations still use it. Why?

IMPORTANT

Organisations still use a DBMS because its benefits, like consistency, security, sharing and integrity, far outweigh its costs.

For large shared data, plain files become unmanageable very quickly: copies multiply, updates clash and searching becomes painful.

A DBMS solves these problems in one place, so the business can trust its data even with thousands of daily transactions.

Modern hardware and cloud options have reduced the cost problem, since a DBMS can now be rented cheaply instead of bought.

The cost of data loss or wrong data, like a wrong bank balance, is far higher than the cost of the DBMS itself.

So almost every serious application today, from banking to e-commerce, runs on a DBMS.

TOPIC 5

Database Users and DBA Roles

5

Naive users: use ready-made applications without knowing SQL, e.g. a bank clerk.

Application programmers: write programs that access the database.

Sophisticated users: write their own queries, e.g. data analysts.

DBA (Database Administrator): the person who manages the whole database.

DBA duties: creating the schema, granting access rights, backup and recovery, performance tuning.

QUESTION BANK

9 Explain the different types of database users. (Mar 2021)

IMPORTANT

Naive users are end users who interact with the database through ready-made application screens without knowing SQL, for example a bank clerk or a railway booking clerk.

Application programmers are software developers who write programs that access the database, for example the team that builds a banking app.

Sophisticated users are skilled people like data analysts who write their own SQL queries to get specific answers from the database.

They understand the database structure well enough to ask ad-hoc questions without a programmer's help.

Specialised users are another category in some books, using custom database applications like a graphics designer using a design database.

The DBA (Database Administrator) is the most privileged user, who administers, secures and maintains the entire database system.

Together these users cover everyone from the casual clerk to the expert who keeps the whole system running.

10 What are the roles and responsibilities of a DBA? (Mar 2021)

IMPORTANT

The DBA (Database Administrator) is the person responsible for managing the entire database system.

The DBA designs the database schema, deciding the structure of tables, keys and relationships.

The DBA creates user accounts and grants access rights, deciding who can read or modify which data.

The DBA takes regular backups and performs recovery after crashes, so no data is ever lost.

The DBA monitors performance and tunes the database, for example by creating indexes, so queries run fast.

The DBA enforces integrity and security rules, making sure invalid data cannot enter the database.

The DBA also manages storage space and plans for growth as data increases.

In short, the DBA is the caretaker of the database, keeping it correct, secure, fast and always available.

11 Who is a database designer? (Dec 2021)

IMPORTANT

A database designer is the person who plans the structure of the database before it is built.

The designer studies what data the organisation needs and how it will be used.

Then the designer identifies the entities (like Student and Course), their attributes, and the relationships between them, usually with an ER diagram.

The designer chooses the keys, removes redundancy through normalization, and writes the table schemas.

In short, the designer decides WHAT the database will store and HOW it will be organised, while the DBA later manages its daily running.

Good design done once saves years of pain: most anomalies and performance problems trace back to design mistakes.

TOPIC 6

Data Models

6

A data model is a way of structuring and organising data in a database.

Hierarchical model: tree structure with parent-child links, like an organisation chart.

Network model: graph structure where a child can have many parents.

Relational model: data stored in tables with rows and columns, e.g. MySQL.

Object-oriented model: data stored as objects with attributes and methods.

Importance: the model decides how data is stored, linked and queried.

QUESTION BANK

12 Explain the different data models with one example each. (Dec 2021)

IMPORTANT

A data model is a way of structuring and organising data in a database, deciding how data is stored, linked and queried.

Hierarchical model: data is arranged in a tree structure with parent-child links, where each child has exactly one parent, for example a company organisation chart with managers and employees.

Network model: data is arranged in a graph structure where a child record can have many parents, allowing many-to-many links, for example a student linked to many courses and each course linked to many students.

Relational model: data is stored in tables with rows and columns linked through keys, for example a Student table in MySQL with columns rollno, name and marks.

Object-oriented model: data is stored as objects that bundle attributes with methods, for example a Student object holding name and marks along with a calculateGrade method.

The relational model is the most widely used today, while the object-oriented model suits complex applications like CAD systems.

13 Why are data models important in DBMS?

IMPORTANT

Data models are important because they decide how data is organised and stored inside the database.

They define how records or tables are linked to each other, such as a student linked to the courses he enrolled in.

They decide how users query and update the data, since each model has its own style of asking questions.

A good data model keeps data consistent, avoids duplication and makes searching fast and simple.

A poor choice of model makes the system slow, hard to change and full of errors.

For example, a billing system fits the relational model, while a social network's friend connections fit a graph-style model.

So choosing the right data model is the first and most important step in database design.

TOPIC 7

Keys in Database Design

Super key: any set of columns that uniquely identifies each row; a candidate key is a minimal super key.

Primary key: the candidate key chosen to identify rows; never null, never repeated.

Alternate key: candidate keys that were not chosen as the primary key.

Foreign key: a column in one table that refers to the primary key of another table.

Composite key: a primary key made of two or more columns; unique key: no duplicates, but one null allowed.

Surrogate key: an artificial key added by the system, e.g. auto-increment id.

QUESTION BANK

14 Define super key, candidate key and primary key with an example.

IMPORTANT

A super key is any set of columns that can uniquely identify each row in a table, for example {rollno, name} or even {rollno, name, marks}.

A candidate key is a minimal super key, meaning no column can be removed from it without losing uniqueness, for example rollno alone or email alone.

A primary key is the candidate key chosen by the designer to actually identify rows in the table, for example rollno.

The primary key has two strict rules: it can never be null and it can never be repeated in two rows.

Consider a Student table with columns rollno, name and email. Here {rollno, name} is a super key, rollno and email are candidate keys, and the designer may choose rollno as the primary key.

Candidate keys that are not chosen become alternate keys.

15 What is a foreign key? Explain with an example. (Model paper)

IMPORTANT

A foreign key is a column in one table that refers to the primary key of another table, creating a link between the two tables.

It guarantees referential integrity, meaning every value in the foreign key must exist as a primary key value in the parent table.

For example, consider Student(rollno, name) where rollno is the primary key, and Marks(rollno, subject, mark).

In the Marks table, rollno is a foreign key that refers to Student(rollno), linking each mark row to the correct student.

Because of the foreign key, nobody can insert marks for a roll number that does not exist in the Student table.

Foreign keys are the main tool for connecting related tables in the relational model.

16 Distinguish between primary key and unique key.

IMPORTANT

A primary key uniquely identifies each row, and there can be only one primary key per table.

The primary key never allows null values and never allows duplicate values.

A unique key also bans duplicate values, but a table can have many unique keys.

The unique key allows one null value, since null is not treated as a duplicate.

For example, in a Student table, rollno can be the primary key, while email can be a unique key.

This means two students cannot share the same email, but a student whose email is not known can have a null there.

In short, use the primary key for the main row identifier and unique keys for other columns that must stay distinct.

17 What are composite and surrogate keys? Give examples.

IMPORTANT

A composite key is a primary key made of two or more columns taken together, used when no single column is unique.

For example, in a Marks table, (rollno, subject) can be the composite primary key, because one student can have many subjects and one subject can belong to many students, but the pair is unique.

A surrogate key is an artificial key added by the system that has no real-world meaning, for example an auto-increment id column.

Surrogate keys are used when no natural column is a good unique identifier, or when natural keys are long and awkward.

For example, a Customer table may use an auto-increment customer_id even though phone numbers exist, because phone numbers can change.

Composite keys come from the data itself, while surrogate keys are invented by the database for convenience.

18 Can a primary key be NULL or duplicate? Justify. (Model paper)

IMPORTANT

No. A primary key can be neither NULL nor duplicate; both are forbidden by definition.

The primary key uniquely identifies each row in a table, so every row must have exactly one distinct, non-empty value.

NULL means unknown or missing, and an unknown value cannot identify a row, so the DBMS rejects NULL in a primary key column.

Duplicates are rejected because two rows with the same key value would be indistinguishable, which breaks the whole purpose of the key.

For example, in Student(rollno, name) with rollno as primary key, two students cannot share rollno 101, and rollno cannot be left blank.

If you try an INSERT with a duplicate or NULL primary key, the DBMS raises a constraint-violation error and the row is not added.

This rule is called entity integrity: it guarantees every row is uniquely and permanently identifiable.

In short: no NULLs, no duplicates - that is what makes a primary key reliable.

TOPIC 8

Table Constraints

8

Primary key constraint: uniquely identifies each row; no nulls allowed.

Foreign key constraint: keeps the link between two tables valid.

Unique constraint: duplicate values are not allowed in the column.

NOT NULL constraint: the column must always have a value.

CHECK constraint: values must satisfy a condition, e.g. age > 0.

QUESTION BANK

19 Explain the different table constraints in DBMS. (Model paper)

IMPORTANT

Table constraints are rules enforced by the DBMS on the data stored in a table, keeping every entry valid.

PRIMARY KEY: uniquely identifies each row; the column cannot be null and cannot have duplicates.

FOREIGN KEY: ensures the link between two tables stays valid by allowing only values that exist as a primary key in the parent table.

UNIQUE: duplicate values are not allowed in the column, though one null is permitted.

NOT NULL: the column must always have a value and cannot be left empty.

CHECK: only values satisfying a given condition are allowed, for example marks between 0 and 100.

DEFAULT: fills in a fixed value automatically when no value is given for the column.

Together these constraints protect the database from wrong, missing or inconsistent data.

20 What is the CHECK constraint? Give an example with SQL.

IMPORTANT

The CHECK constraint allows only those values in a column that satisfy a given condition, rejecting everything else.

It is used to enforce business rules directly in the database, so no program can store invalid data by mistake.

For example, marks in a Student table must always be between 0 and 100.

This rule can be written in SQL while creating the table:

```
CREATE TABLE Student (
rollno INT PRIMARY KEY,
name VARCHAR(50) NOT NULL,
marks INT CHECK (marks >= 0 AND marks <= 100)
);
```

If anyone tries to insert marks like 150 or -10, the DBMS rejects the row with an error.

This keeps the data correct without every application having to check the rule itself.

21 What is referential integrity constraint? (Oct 2019)

IMPORTANT

Referential integrity is the rule that a foreign key value must always match an existing primary key value in the referenced table, or be NULL.

It keeps relationships between tables valid: you cannot point to a row that does not exist.

For example, in Enroll(stud_id, course_id) where stud_id references Student(rollno), every stud_id in Enroll must exist as a rollno in Student.

If you try to insert an enrollment for rollno 999 when no such student exists, the DBMS rejects it.

It also controls deletes and updates: deleting a student who still has enrollments is blocked (or cascaded), so no orphan rows are left behind.

The constraint is declared with FOREIGN KEY ... REFERENCES in the CREATE TABLE statement.

Without it, the database could fill up with dangling references pointing nowhere.

So referential integrity guarantees that links between tables always point to real, existing rows.